Import Required Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Visualization style
sns.set(style="whitegrid")
```

Load the Dataset

In [2]:
```
import os
os.path.isfile(r"C:\data\Netflix.csv")
```

Out[2]:
```
False
```

In [3]:
```
import os
os.path.isfile(r"C:\Users\Aryan\Downloads\Netflix")
```

Out[3]:
```
False
```

In [4]:
```
import os
os.path.isfile(r"C:\Users\Aryan\OneDrive\เอกสาร\data science\Netfli
```

Out[4]:
```
True
```

In [5]:
```
df = pd.read_csv(r"C:\Users\Aryan\OneDrive\เอกสาร\data science\Netf
```

Check First 5 Rows (HEAD)

In [6]:
```
df.head()
```

Out[6]:

	Sl.No	Title	Description	Genres	Cast	
0	1	Alchemy of Souls	A powerful sorceress in a blind woman's body e...	International TV Shows,\uf\uf\uf

Out[7]: (6069, 13)

Column Names

In [8]:
```
df.columns
```

Out[8]:
```
Index(['Sl.No', 'Title', 'Description', 'Genres', 'Cast', 'Directo
r',
       'Production Country', 'Added to Netflix On', 'Year', 'Serie
s',
       'Duration(in Minutes)', 'Season', 'Rating'],
      dtype='object')
```

Detecting spelling issues

Renaming columns if needed

Basic Dataset Information

In [9]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6069 entries, 0 to 6068
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Sl.No                 6069 non-null   int64
 1   Title                 6068 non-null   object
 2   Description           6068 non-null   object
 3   Genres                6068 non-null   object
 4   Cast                  5479 non-null   object
 5   Director              3966 non-null   object
 6   Production Country    1878 non-null   object
 7   Added to Netflix On   6069 non-null   object
 8   Year                  6068 non-null   float64
 9   Series                6065 non-null   object
 10  Duration(in Minutes)  3884 non-null   float64
 11  Season                2185 non-null   object
 12  Rating                5014 non-null   float64
dtypes: float64(3), int64(1), object(9)
memory usage: 616.5+ KB
```

Check:

Data types (object, int, float)

Null counts

Memory usage

📌 You'll notice:

Dates are object

Duration mixed (minutes + seasons)

Check Missing Values (ISNULL)

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
Sl.No                     0
Title                     1
Description               1
Genres                    1
Cast                    590
Director               2103
Production Country     4191
Added to Netflix On       0
Year                      1
Series                    4
Duration(in Minutes)   2185
Season                 3884
Rating                 1055
dtype: int64
```

Percentage of Missing Values

In [11]:
```
(df.isnull().sum() / len(df)) * 100
```

Out[11]:
```
Sl.No                   0.000000
Title                   0.016477
Description             0.016477
Genres                  0.016477
Cast                    9.721536
Director               34.651508
Production Country     69.055858
Added to Netflix On     0.000000
Year                    0.016477
Series                  0.065909
Duration(in Minutes)   36.002636
Season                 63.997364
Rating                 17.383424
dtype: float64
```

📌 This tells you:

Which columns need cleaning

Which can be ignored

Handle Missing Values (Cleaning) --> Fill Missing Ratings with Mean

In [12]:
```
df['Rating'] = df['Rating'].fillna(df['Rating'].mean())
```

Fill Missing Text Columns with "Unknown"

In [13]:
```
text_cols = ['Director', 'Cast', 'Production Country', 'Genres']

for col in text_cols:
    df[col] = df[col].fillna("Unknown")
```

Verify Again

In [14]:
```
df.isnull().sum()
```

Out[14]:
```
Sl.No                      0
Title                      1
Description                1
Genres                     0
Cast                       0
Director                   0
Production Country         0
Added to Netflix On        0
Year                       1
Series                     4
Duration(in Minutes)    2185
Season                  3884
Rating                     0
dtype: int64
```

Remove Duplicate Rows

In [15]:
```
df.duplicated().sum()
```

Out[15]:
```
np.int64(0)
```

In [16]:
```
df = df.drop_duplicates()
```

Date Cleaning

In [17]:
```
df['Added to Netflix On'] = pd.to_datetime(
    df['Added to Netflix On'], errors='coerce'
)
```

```
C:\Users\Aryan\AppData\Local\Temp\ipykernel_9140\1299539873.py:1: Us
erWarning: Could not infer format, so each element will be parsed in
dividually, falling back to `dateutil`. To ensure parsing is consist
ent and as-expected, please specify a format.
  df['Added to Netflix On'] = pd.to_datetime(
```

pd.to_datetime() converts unparsable date string to NaT(NOT A TIME)

Extract Year Added

WHY DO WE NEED THIS?

Because:

You cannot easily group by full dates

Year helps in:

trend analysis

bar charts

time comparison

In [18]:
```
df['Added_Year'] = df['Added to Netflix On'].dt.year
```

Duration Cleaning

convert duration into numeric minutes

Because i type a wrong column name which is "Duration(in Minutes)" so this error found.

In [19]:
```
df.columns
```

Out[19]:
```
Index(['Sl.No', 'Title', 'Description', 'Genres', 'Cast', 'Directo
r',
       'Production Country', 'Added to Netflix On', 'Year', 'Serie
s',
       'Duration(in Minutes)', 'Season', 'Rating', 'Added_Year'],
      dtype='object')
```

In [20]:
```
def clean_duration(value):
    if isinstance(value, str):
        if 'SEASON' in value:
            return np.nan
        else:
            return int(value)
```

```
    return value
df['Duration(in Minutes)'] = df['Duration(in Minutes)'].apply(clean
```

Statistical Summary

In [21]:
```
df.describe()
```

Out[21]:

	Sl.No	Added to Netflix On	Year	Duration(in Minutes)	Ra
count	6069.000000	6069	6068.000000	3884.000000	6069.00
mean	3035.000000	2020-04-17 12:31:40.543747072	2016.759064	98.742019	6.51
min	1.000000	2013-08-02 00:00:00	1925.000000	3.000000	1.40
25%	1518.000000	2019-03-08 00:00:00	2016.000000	85.000000	6.00
50%	3035.000000	2020-09-01 00:00:00	2019.000000	98.500000	6.51
75%	4552.000000	2021-08-27 00:00:00	2020.000000	115.000000	7.20
max	6069.000000	2022-06-19 00:00:00	2022.000000	312.000000	9.50
std	1752.113723	NaN	6.919702	30.785851	1.06

Column Type

Why it appears

Sl.No int Numeric

Year float Numeric

Duration(in Minutes) float Numeric

Rating float Numeric

Added_Year float Numeric

Added to Netflix On datetime Shows limited stats

📌 Datetime columns show:

min

max

mean (as timestamp)

❌ WHY OBJECT COLUMNS ARE NOT SHOWN

These columns are object type:

Title

Description

Genres

Cast

Director

Production Country

Series

Season

📌 Statistics like mean, min, std make no sense for text.

So pandas hides them.

HOW TO SEE OBJECT (TEXT) COLUMNS SUMMARY

🔹 Option 1: Object-only summary df.describe(include='object')

This shows:

count

unique

top

freq

(Which is exactly the table you shared earlier 👌)

🔹 Option 2: See EVERYTHING together df.describe(include='all')

⚠️ This will show:

numeric stats for numbers

categorical stats for text

many NaN values (normal)

For categorical data:

In [22]:
```
df.describe(include='object')
```

Out[22]:

	Title	Description	Genres	Cast	Director	Production Country	Seri
count	6068	6068	6069	6069	6069	6069	60
unique	6004	6020	449	5231	2998	189	
top	Wild Dog	An unhappy Mouna plots to divorce her talkativ...	Stand-Up Comedy	Unknown	Unknown	Unknown	T M
freq	4	5	360	590	2103	4191	27

🔍 WHAT THIS TABLE IS

This is categorical summary, telling you:

how many values exist

how diverse the data is

which value appears the most

🧱 MEANING OF EACH ROW 1️⃣ count

👉 Number of non-null (non-empty) values

Example:

Title = 6068 → 1 title missing

Series = 6065 → 4 missing

Season = 2185 → MOST entries are empty

📌 Insight: Only a small fraction of Netflix content has seasons → most are movies.

2️⃣ unique

👉 Number of distinct values

Example:

Genres = 449 → 449 genre combinations

Production Country = 189 → very global

Director = 2998 → many different creators

📌 Insight: Netflix has high content diversity.

3️⃣ top

👉 Most frequently occurring value (mode)

Example:

Genre → Stand-Up Comedy

Cast → Unknown

Director → Unknown

Country → Unknown

Season → 1 SEASON

📌 Insight: Metadata is incomplete + Stand-Up Comedy is heavily produced.

4️⃣ freq

👉 How many times the top value appears

Example:

Stand-Up Comedy → 360 times

Unknown Director → 2103 times

Unknown Country → 4191 times

TV-MA → 2733 times

1 SEASON → 1450 times

📌 Insight: Netflix has:

Many single-season shows

Large missing metadata (director, cast, country)

📊 COLUMN-WISE EXPLANATION 🎬 Title

6004 unique titles

Some titles repeated (remakes, seasons)

📌 Repetition ≠ duplication error

📝 Description

Very diverse descriptions

Repetition due to similar marketing summaries

🎭 Genres

449 unique combinations

Stand-Up Comedy dominates

📌 Netflix pushes comedy for engagement

👥 Cast & 🎥 Director

“Unknown” dominates

Indicates:

Incomplete data

Not all titles list creators

📌 Data quality limitation

🌍 Production Country

“Unknown” is highest (4191)

📌 Country-based analysis must be done carefully

📺 Series / Rating

TV-MA is most common

📌 Netflix focuses on mature audience content

📦 Season

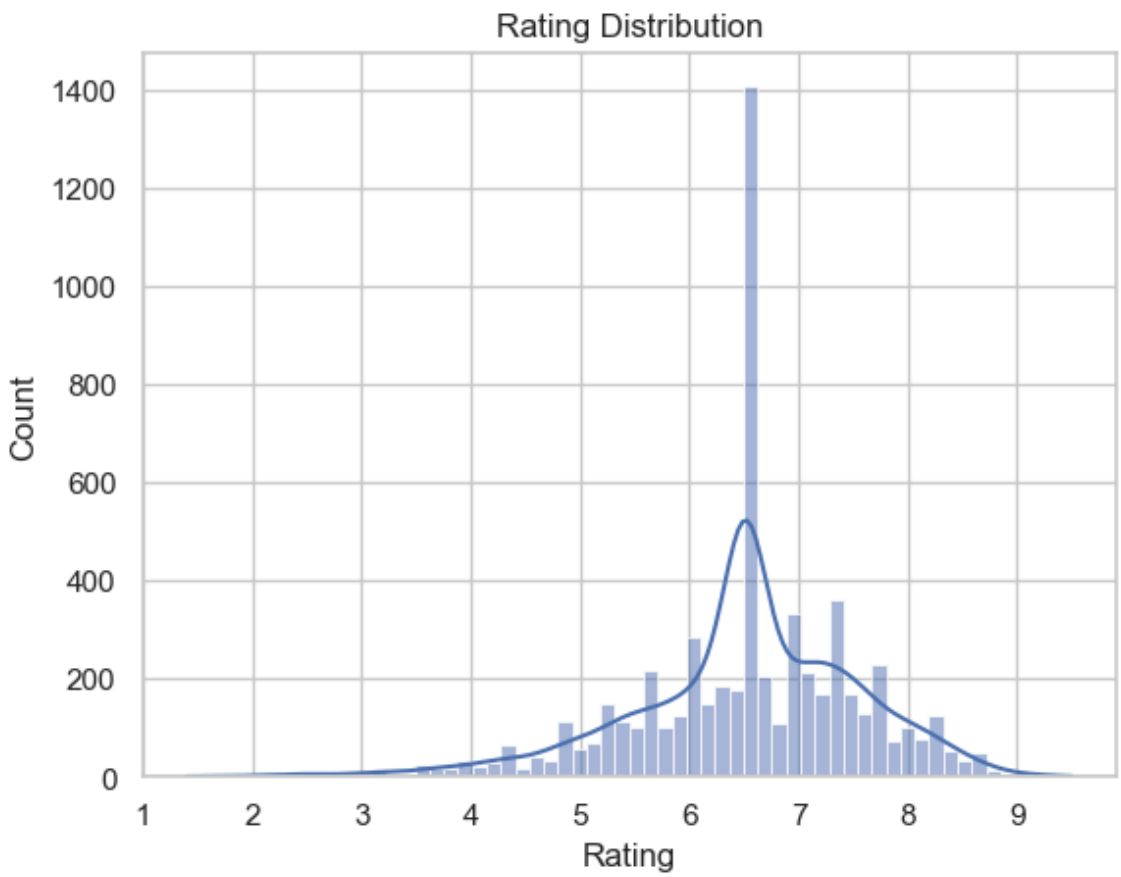
Only 2185 entries

Most are 1 SEASON

📌 Limited-series strategy

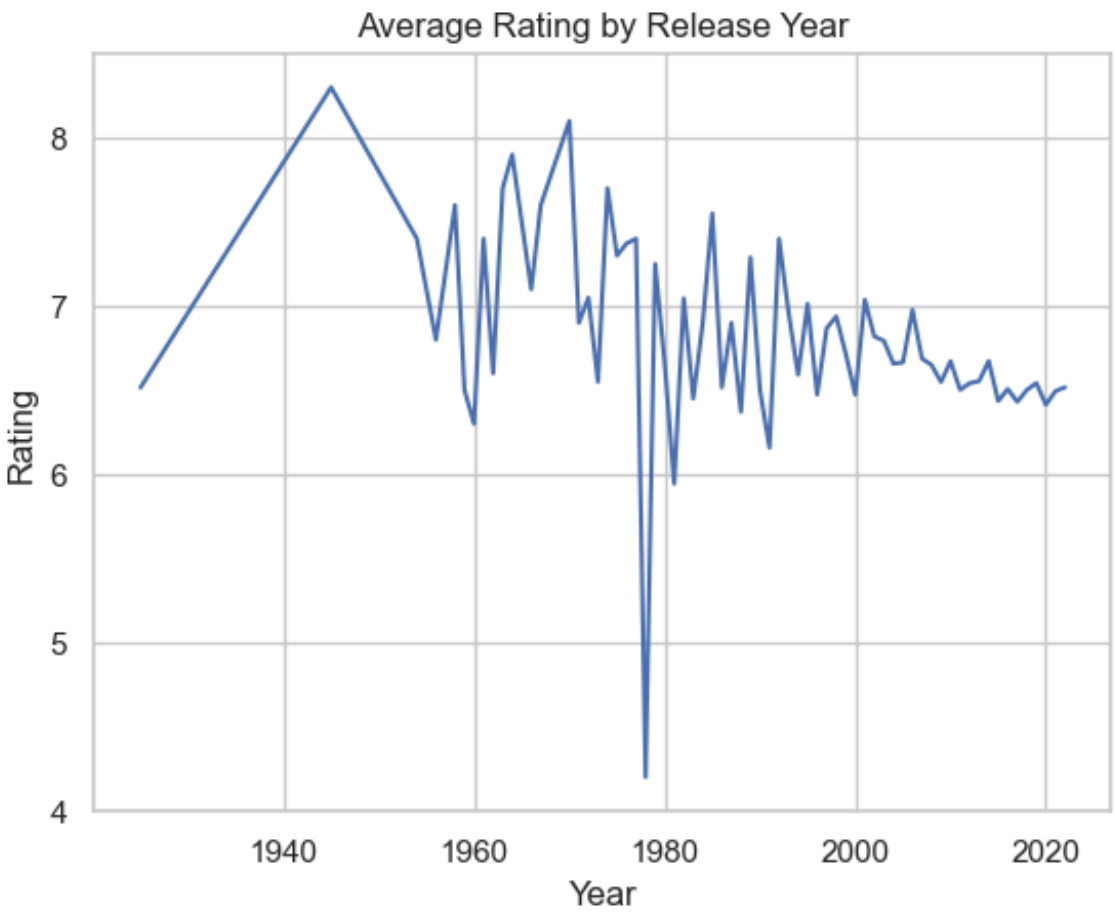
Analysis (Single Column) Rating Distribution

In [23]:
```
plt.figure()
sns.histplot(df['Rating'], kde=True)
plt.title("Rating Distribution")
plt.show()
```

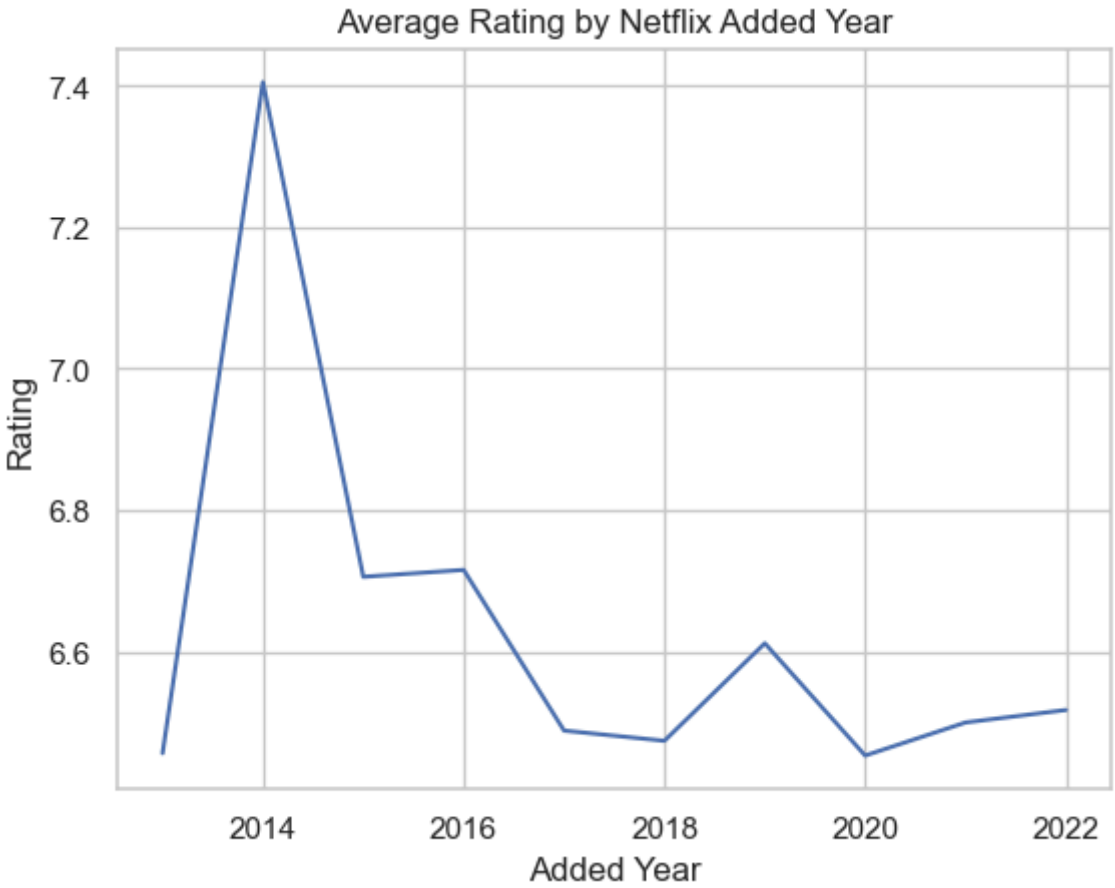


Analysis Average Rating by Release Year

In [24]:
```
avg_rating_year = df.groupby('Year')['Rating'].mean()

plt.figure()
avg_rating_year.plot()
plt.title("Average Rating by Release Year")
plt.xlabel("Year")
plt.ylabel("Rating")
plt.show()
```



In [25]:
```
avg_rating_added_year = df.groupby('Added_Year')['Rating'].mean()

plt.figure()
avg_rating_added_year.plot()
plt.title("Average Rating by Netflix Added Year")
plt.xlabel("Added Year")
plt.ylabel("Rating")
plt.show()
```



This graph shows:

📅 For each year, what was the average rating of content that Netflix added in that year

X-axis (Added Year): Year when the title was added to Netflix (2013 → 2022)

Y-axis (Rating): Average audience rating of those titles

📊 HOW TO READ THE TREND (YEAR BY YEAR) 🔹 2013–2014

Sharp increase

2014 has the highest average rating (~7.4)

📌 Meaning: Early Netflix additions were high-quality, carefully selected content.

🔹 2015–2016

Ratings start to decline (~6.7)

📌 Netflix begins scaling content volume, slight quality drop.

🔹 2017–2018

Lowest average ratings (~6.48–6.5)

📌 Period of content saturation and experimentation.

🔹 2019

Small recovery (~6.6)

📌 Improvement in content selection.

🔹 2020

Drop again (~6.45)

📌 COVID-era mass production → quantity over quality.

🔹 2021–2022

Slight upward stabilization (~6.5+)

📌 Netflix begins quality control & optimization.

🧠 KEY INSIGHTS (VERY IMPORTANT)

1️⃣ Early Netflix strategy focused on quality

2️⃣ Rapid expansion reduced average ratings

3️⃣ Recent years show stabilization, not growth

4️⃣ Netflix is now in a mature optimization phase

Genre Analysis

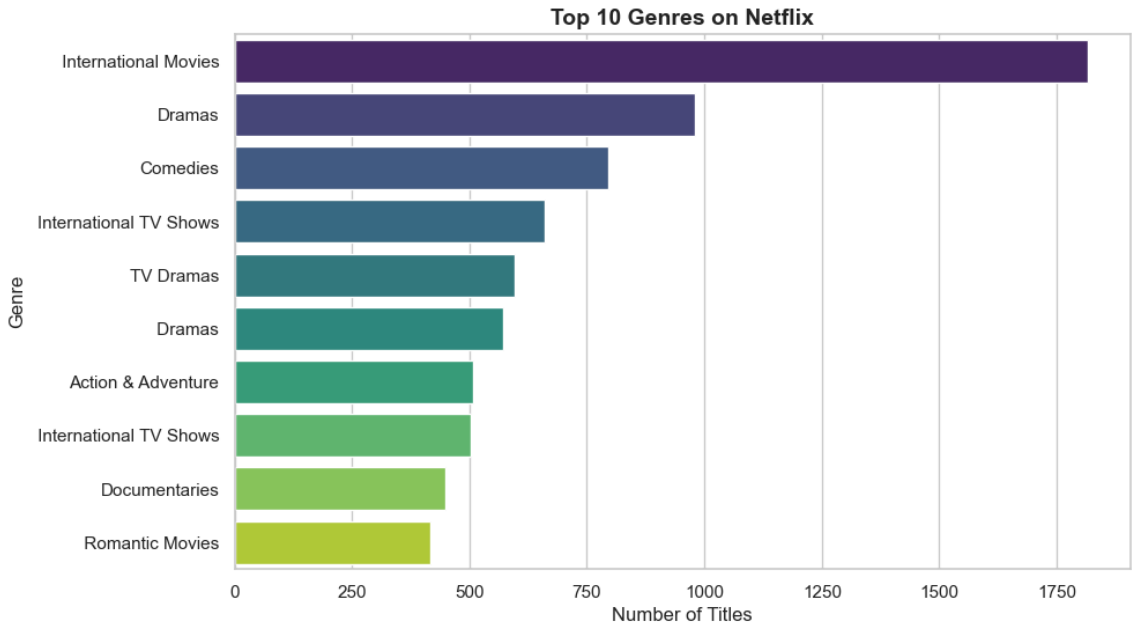
re-create top_genres before plotting.

In [26]:
```
# Step 1: Split and explode genres
genres = df['Genres'].str.split(',').explode()

# Step 2: Get top 10 genres
top_genres = genres.value_counts().head(10)
```

In [30]:
```
plt.figure(figsize=(10, 6))

sns.barplot(
    x=top_genres.values,
```

```
    y=top_genres.index,
    palette="viridis"
)

plt.title("Top 10 Genres on Netflix", fontsize=14, fontweight="bold
plt.xlabel("Number of Titles")
plt.ylabel("Genre")
plt.show()
```

```
C:\Users\Aryan\AppData\Local\Temp\ipykernel_9140\3341640674.py:3: Fu
tureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set `legend
=False` for the same effect.

  sns.barplot(
```



1️⃣ International Movies dominate Netflix

International Movies have the highest number of titles by a large margin.

👉 Insight: Netflix strongly focuses on global content, not just Hollywood. This supports Netflix's strategy of regional & multilingual expansion.

2️⃣ Drama is the most popular content theme

Dramas appear multiple times and rank very high.

👉 Insight: Viewers prefer story-driven, emotional content, both in movies and TV shows. Drama acts as a core genre across formats.

3️⃣ Comedy and TV content have strong presence

Comedies, TV Dramas, and International TV Shows all rank in the top.

👉 Insight: Netflix invests heavily in binge-watchable TV series and light-hearted content.

4️⃣ Action & Adventure and Documentaries are moderate

These genres are present but not dominant.

👉 Insight: They serve niche audiences compared to drama and comedy.

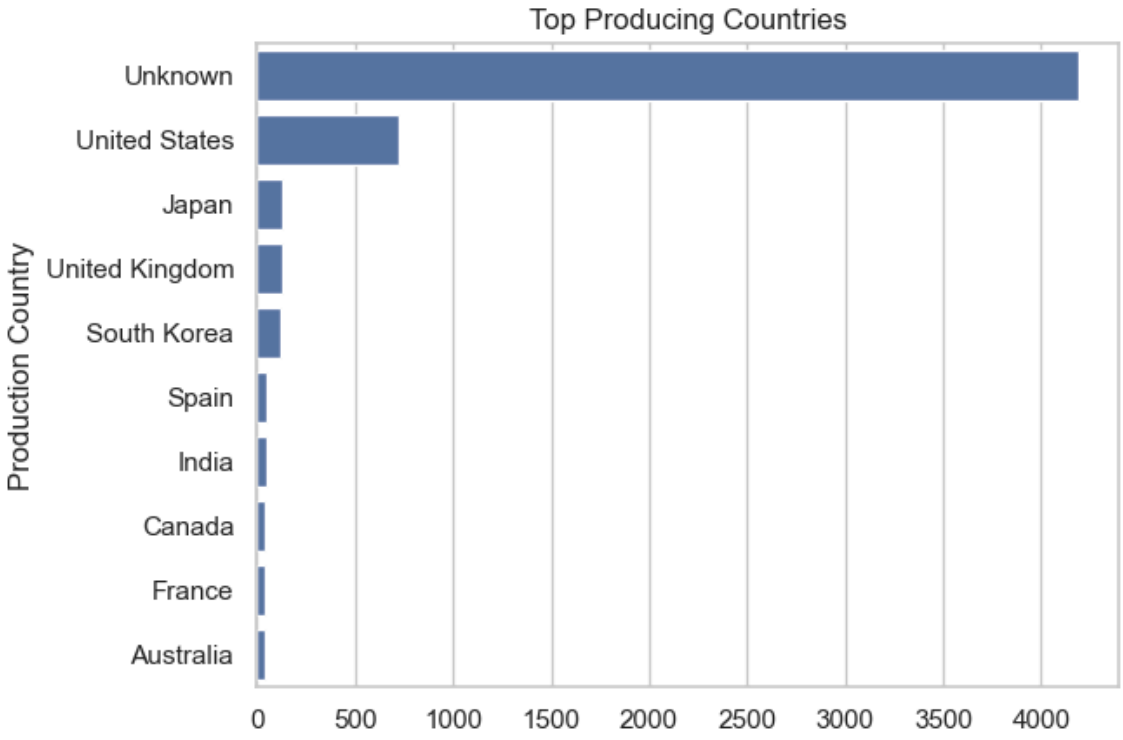
5️⃣ Romantic Movies are comparatively fewer

Romantic Movies appear at the bottom of top 10.

👉 Insight: Romance is usually combined with other genres rather than standing alone.

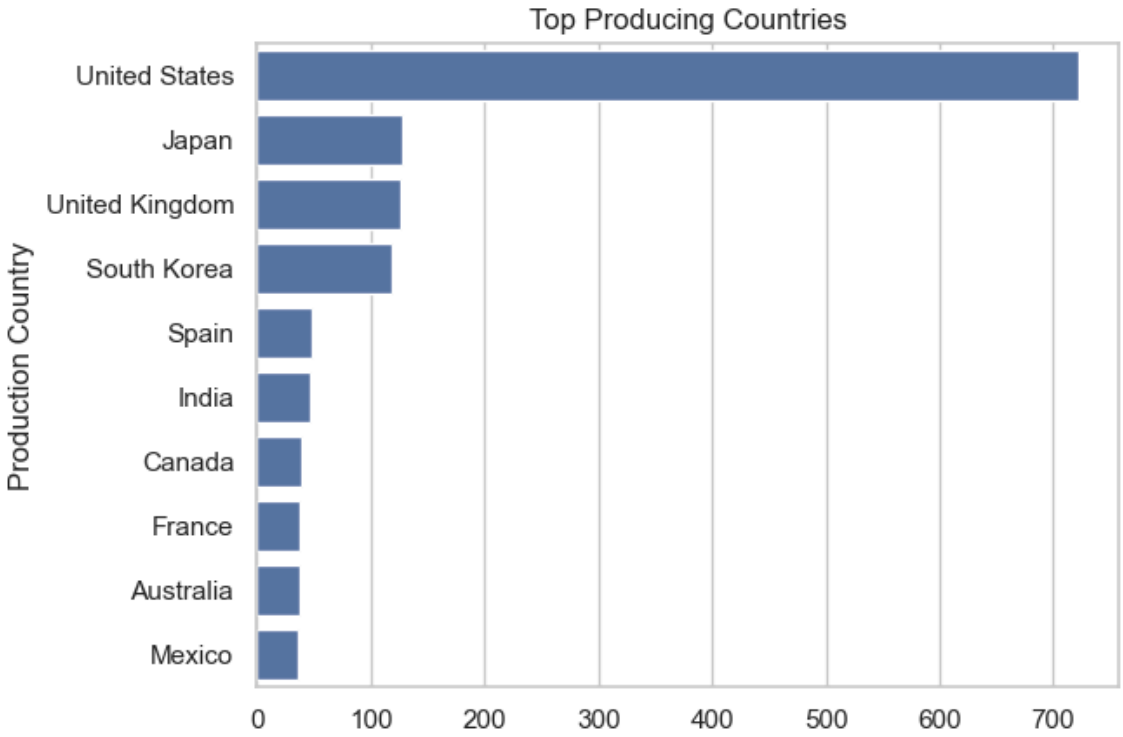
Country Analysis

In [31]:
```
top_countries = df['Production Country'].value_counts().head(10)

plt.figure()
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title("Top Producing Countries")
plt.show()
```



In [32]:
```
df['Production Country'] = df['Production Country'].replace('Unknow
```

In [51]:
```
top_countries = df['Production Country'].value_counts().head(10)

plt.figure()
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title("Top Producing Countries")
plt.show()
```



In [52]:
```
df['Production Country'].isna().sum()
```

Out[52]: np.int64(4191)

❌ Before Production Country

Unknown United States Unknown India Unknown

value_counts() → "Unknown" appears on top

Graph becomes misleading

✅ After Production Country

NaN United States NaN India NaN

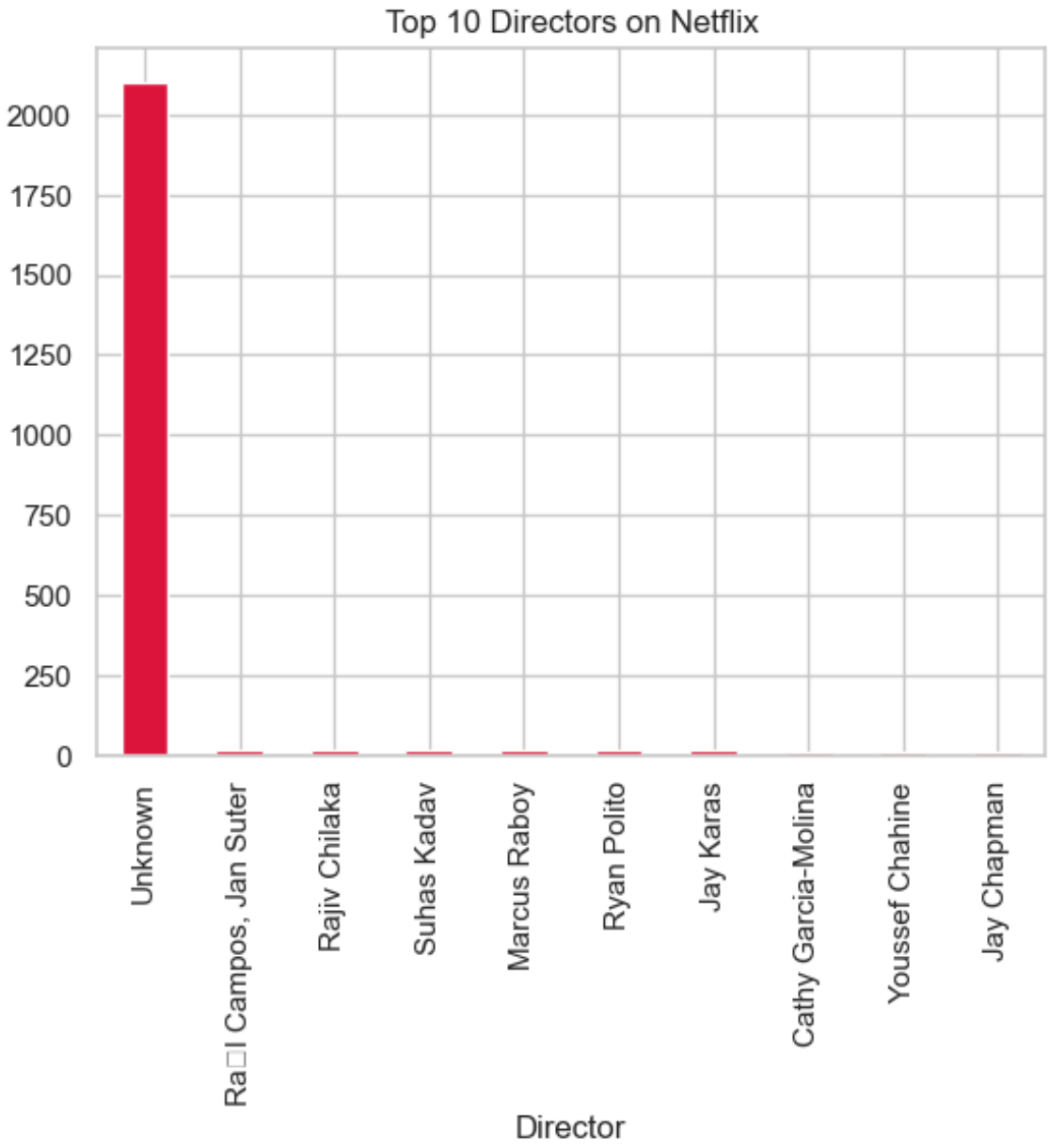
Missing values are ignored automatically

Graph shows real countries only

In [41]:
```
# Convert date column
df['Added to Netflix On'] = pd.to_datetime(df['Added to Netflix On'

# Extract year
df['Added_Year'] = df['Added to Netflix On'].dt.year

# Calculate Content Age
df['Content_Age'] = df['Added_Year'] - df['Year']
```

In [49]:
```
top_directors = df['Director'].value_counts().head(10)
top_directors.plot(kind='bar', color='crimson')
plt.title("Top 10 Directors on Netflix")
plt.show()
```

```
c:\Users\Aryan\anaconda3\Lib\site-packages\IPython\core\pylabtools.p
y:170: UserWarning: Glyph 65533 (\N{REPLACEMENT CHARACTER}) missing
from font(s) Arial.
  fig.canvas.print_figure(bytes_io, **kw)
```



1️⃣ Data Quality Insight

👉 A large number of missing director values shows the dataset is incomplete or scraped automatically.

Write this:

The dataset contains missing metadata, which is common in real-world big data collected from web sources.

2️⃣ Metadata Limitation Insight

👉 OTT platforms sometimes do not publicly display director information, especially for:

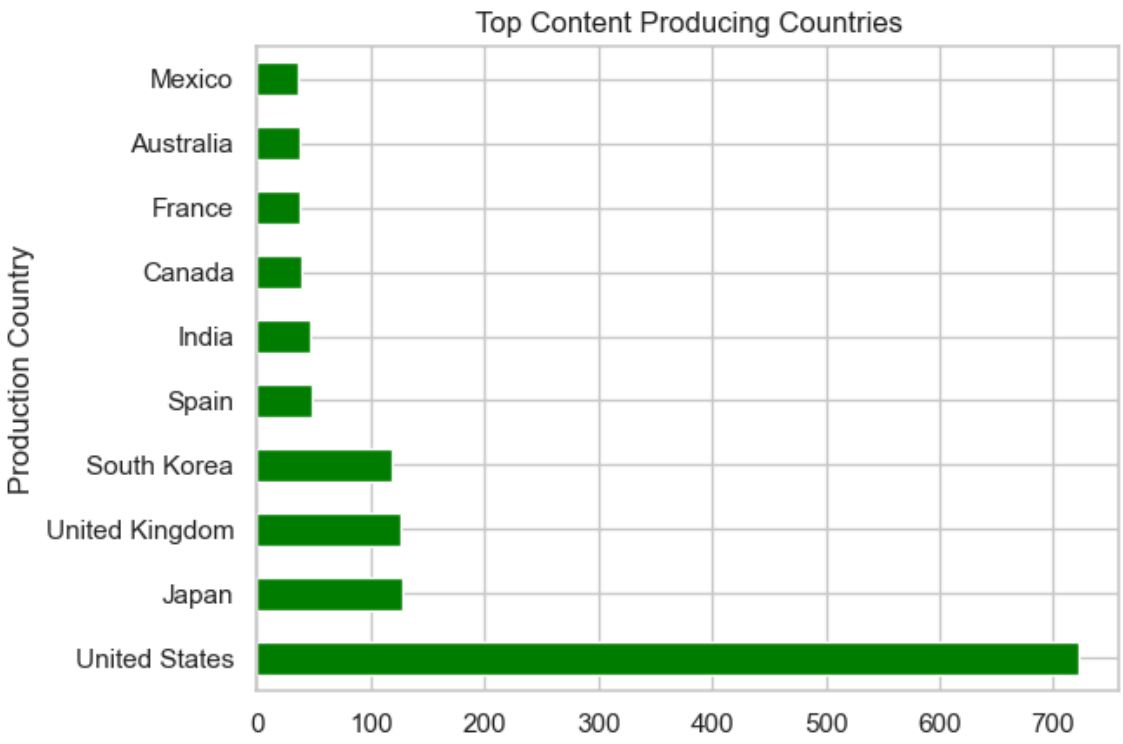
TV shows

International content

Regional movies

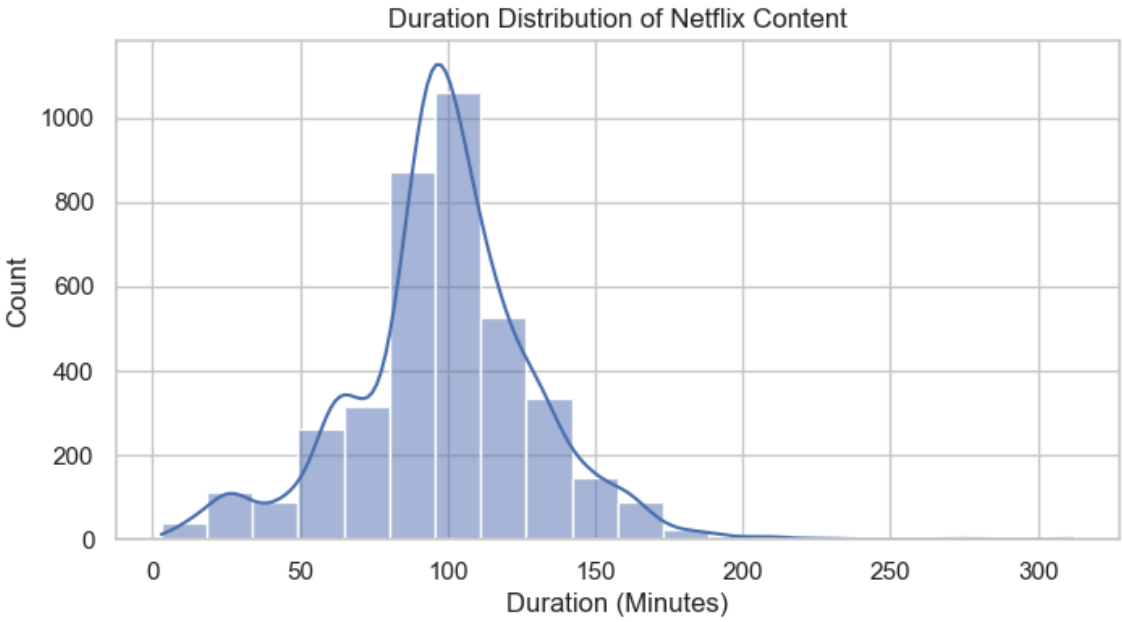
Meaning: Metadata is not always standardized.

In [50]:
```
top_country = df['Production Country'].value_counts().head(10)
top_country.plot(kind='barh', color='green')
plt.title("Top Content Producing Countries")
plt.show()
```



Duration Distribution (Viewer Engagement Insight)

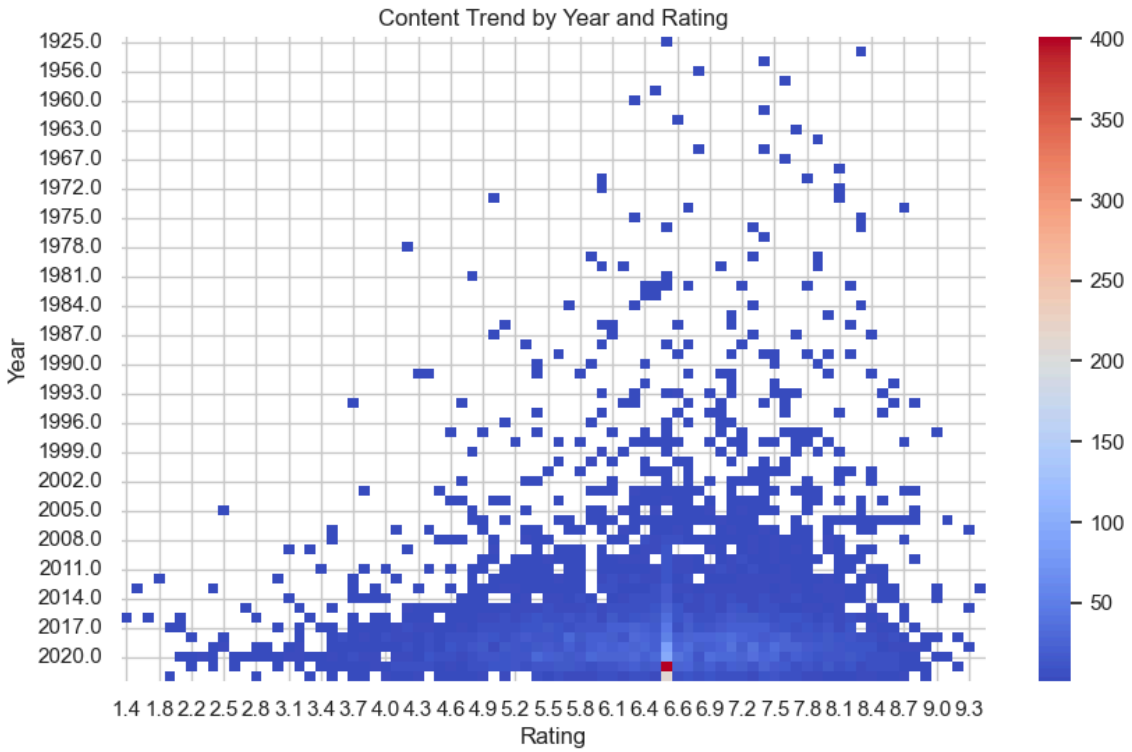
In [52]:
```
plt.figure(figsize=(8,4))
sns.histplot(df['Duration(in Minutes)'], bins=20, kde=True)
plt.title("Duration Distribution of Netflix Content")
plt.xlabel("Duration (Minutes)")
plt.show()
```



Insight

Most content is between 60–120 minutes, indicating standard movie length preference.

In [54]:
```
pivot = pd.pivot_table(df, values='Sl.No', index='Year', columns='R

plt.figure(figsize=(10,6))
sns.heatmap(pivot, cmap='coolwarm')
plt.title("Content Trend by Year and Rating")
plt.show()
```



Insight

➡️ Mature-rated content has increased in recent years, showing shift in audience demand.

In [32]:
```
df.to_csv("cleaned_netflix_data.csv", index=False)
```

CONCLUSION

1. This project analyzed Netflix data using Python and visualization techniques.
2. Netflix content production has grown rapidly over the years.
3. Few countries dominate global content creation.
4. Netflix targets mature audiences with high-rated content.
5. Most content follows standard movie duration patterns.
6. Movies dominate, but series show increasing popularity.
7. Data visualization helped uncover hidden business trends.

8. The project improved skills in data cleaning, analysis, and visualization.

9. Such analysis helps OTT platforms optimize content strategy and audience targeting.